SUBMISSION PLAN · 7–10 NOVEMBER 2026

3-Day Execution Plan

From a working local workbench to a downloadable, demonstrable security tool

Document	Value
Version	1.0 (execution baseline)
Date	7 November 2026
Submission window	3 days · target 10 November 2026
Platform	Fedora 43 first; Ubuntu/Kali/Windows only if time remains
Core constraint	Static target analysis only; samples are compiled, never executed
Related document	ReverseShield_Project_Report_and_Plan.docx

How to read this document

This is an execution document, not a marketing description. Every task has an output and an acceptance check. The project remains a static-analysis workbench; protected samples are compiled but never executed. Security claims are limited to observed artifact changes.

Contents

Page numbers refer to this PDF build. The Word file uses the same section order.

Section	Page
Document control	03
1. Executive summary	03
2. Baseline: what already exists	04
3. Gaps, risks, and the cut line	04
4. Scope and non-goals	05
5. Priorities: what must ship	06
6. Three-day schedule	06
7. P0 playbooks	08
8. UI upgrade design	10
9. Packaging: download and run on Fedora	11
10. SAST, DAST, and data-leak prevention	13
11. Evidence pack and demo script	15
12. Submission checklist	15
13. References	16
Appendix A — Commands and file map	16

Document control

Item	Value
Document	ReverseShield — 3-Day Execution Plan
Purpose	Turn the current v0.2 workbench into a submittable, testable, downloadable project
Audience	Project author, supervisor, evaluation committee
Status	Execution plan; not a claim that the work is complete
Assumptions	8 focused hours per day; Fedora 43 test machine; no target execution; local-only DAST
Change rule	P0 tasks are fixed. P1 tasks may be reduced. P2 tasks are cut first.

The one-sentence goal

A reviewer downloads one Fedora bundle, runs one command, clicks one demo, sees a before/after comparison, and exports a sanitized evidence report — without installing anything system-wide and without executing a target.

1. Executive summary

ReverseShield already works as a local static-analysis workbench. It discovers installed Flatpak applications, unpacks RPM payloads as data, accepts folders and ELF uploads, runs six bounded static tools (including Ghidra 12.2 headless), streams live events, stores evidence under runs/<job_id>/, and compares a bundled sample before and after a transformation, plus a binary metadata hygiene profile for a copy of any selected ELF. The current suite has **54 tests**, with one opt-in Ghidra test skipped by default.

The submission gap is not 'more features'. It is **packaging, usable flow, honest protection evidence, and security analysis**. The current demo protection is visibly weak: the build keeps debug symbols, the key is present twice in .data, only two integer literals are blinded, the opaque guard is removed by Ghidra, and the aggregate comparison counts hide all of this. The current report also contains local paths and the demo key.

This plan uses three days to produce four deliverables: (1) a Fedora one-command bundle, (2) a one-click demo flow with a before/after dashboard, (3) a stronger but still demonstration-grade protection build with sanitized reporting, and (4) SAST/DAST evidence with a clear limitations section. The protection claim will remain deliberately modest: **observed artifact changes, not resistance to reverse engineering**.

Outcome	Definition of done	Proof to capture
Downloadable app	`install.sh` + `run.sh` on Fedora 43; no root; browser opens	Terminal transcript, installed desktop entry, smoke test
One-click demo	Select Level 5 → analyze → protect → compare → export in one flow	Screen recording or annotated screenshots
Protection evidence	Release build has no debug/symbols; key differs per build; policy table encoded	`readelf`, `nm`, `strings`, Ghidra before/after table
Security analysis	SAST + ZAP baseline + report redaction; no high/medium findings unaddressed	Semgrep/Bandit/ZAP JSON or HTML, sanitized report
Submission pack	Two Word documents, checksums, demo script, limitations	`CHECKSUMS.txt`, PDF page check, rehearsal timing

2. Baseline: what already exists

Do not rebuild the backend. The current project is a working prototype with a coherent architecture and real evidence; the next three days should extend it, not replace it.

Area	Current state	Evidence
Backend	Python standard-library HTTP server, loopback-only, per-session token, asynchronous jobs	`app.py`, `http_upgrade.py`, `workflow.py`; 54 tests
Analysis	`file`, `readelf`, `objdump`, `strings`, `nm`, Ghidra headless; bounded output and timeouts	Level 5 Ghidra run: 6.34 s; 13 functions, 10 pseudocode bodies
Discovery	Flatpak list/targets, local folder, ELF, RPM payload, uploaded ELF/RPM	9 Flatpak apps; EasyEffects: 11 ELF targets
Protection	GNU `strip --strip-unneeded`; demo constant blinding + opaque guard on sample copies; binary metadata hygiene on any ELF copy	Level 5 strip/protect/hygiene reports with SHA-256 hashes
UI	Operate / Compare / Monitor / Read Me; live event stream; downloads	Dark cyan interface; screenshots in `docs/assets/`
Storage	`runs/<job_id>/` with snapshots, events, report, Ghidra exports, protected artifacts	15 job folders, 51 MB at baseline
Packaging	Manual `python3 app.py`; no installer, desktop entry, or release archive	Gap — primary Day 1–3 work
Security analysis	No integrated SAST/DAST; preliminary Bandit/Semgrep/detect-secrets only	Bandit: 18 low, 0 medium/high; Semgrep: 0; detect-secrets: 0

Baseline evidence from the current reports

Level 5 symbol removal: 16,088 → 10,784 bytes; symbols in nm: 29 → none; strings: 132 → 61; Ghidra functions: 13 → 11. Level 5 demo blinding: 16,088 → 16,528 bytes; .text 443 → 581 bytes; policy_allows 86 → 161 bytes; XOR instructions 3 → 6; rs_key references 0 → 13; Ghidra still recovers the policy logic. These are the numbers the plan must improve and explain.

3. Gaps, risks, and the cut line

3.1 Protection gaps that must be fixed

Gap	Observed evidence	Why it matters
Debug info and symbols remain	Protected file says `with debug_info, not stripped`; `policy_allows`, `policy_reason`, `main` are named	The analyst gets the map for free; the 'protected' artifact leaks more than a stripped one
Key is exposed twice	`.data` contains `b9 79 37 9e` at `0x403004` and `0x403008`; report contains `"key": "0x9E3779B9"`	Anyone can XOR the constants back; the key is not secret
Only two literals blinded	`7u` and `0x80000000u`; policy values 1/2/3/99 remain in `policy_allows`	The core policy table is readable; the transformation is mostly cosmetic
Guard is not opaque	`rs_key ^ rs_key`; Ghidra marks the block unreachable in `main` and removes it from pseudocode	Adds bytes but not analytical cost
Fixed key	Same `0x9E3779B9` for every build	Stable signature across samples; no per-build diversity

Gap	Observed evidence	Why it matters
Aggregate metrics mislead	Symbols 111→120 mixes Ghidra labels; strings 132→134 is debug/path noise; functions 14→14	A reviewer cannot see whether protection did anything

3.2 UX and distribution gaps

Gap	Current behaviour	Target for Day 3
No one-click flow	User must discover, select, choose tools, analyze, then run protection separately	One button runs the full Level 5 demo with safe defaults
Compare tab is empty until a job is opened	It shows a hint but no action	Offer 'Open latest comparison' and 'Run demo'
Monitor rows are cramped	Label, operation, status and ID run together	Grid columns with clear spacing and truncation
No storage visibility	User must run `du` in a terminal	Jobs table shows size, total, delete action
Only full JSON export	Report contains paths, commands, logs and key	Sanitized export by default; raw export behind a warning
No installer	Requires manual Python start and pre-existing tools	Fedora tarball with `install.sh`, `run.sh`, desktop entry

3.3 Risks and contingency

Risk	Likelihood	Impact	Contingency
Protection v2 breaks the sample build	Medium	High	Keep current transformer behind a flag; revert to artifact hygiene + report redaction if tests fail
ZAP image cannot run on Fedora	Medium	Medium	Run ZAP desktop manually; keep `curl` API tests and document the container command as tested-on-other-host
Ghidra is slow on EasyEffects	High	Low	Demo Level 5; show EasyEffects discovery only, with measured limits
Time slips on UI polish	High	Medium	Freeze P0 UI at a clean stepper; cut animations and P2
Word layout issues	Medium	Low	Convert to PDF, inspect cover + dense tables + final page, leave a 1-hour buffer
Scope creep into real obfuscation	High	High	Restate the claim: demonstration-grade evidence, not a protection product

Cut line

If a task is not done by the Day 2 checkpoint, cut P2 first (Podman/AppImage, authenticated ZAP API scan, anti-tamper, dynamic binary analysis). Never cut P0: release artifact hygiene, sanitized report, one-click flow, installer, tests.

4. Scope and non-goals

In scope (3 days)	Out of scope (state explicitly)
Fedora 43 one-command install and run	Windows, macOS, AppImage, Snap, Flatpak packaging (future)

In scope (3 days)	Out of scope (state explicitly)
Static analysis of selected ELF targets	Executing, emulating, or dynamically tracing target binaries
Source protection on bundled samples; binary metadata hygiene on one selected ELF copy	Binary code rewriting, packing, or protecting third-party apps
SAST on project code; DAST on the local workbench	Scanning third-party sites or apps without authorization
Report sanitization and secret/PII checks	A full DLP product or enterprise data classification
Ghidra, binutils, and ZAP baseline evidence	Full active exploitation or fuzzing of the host
Ubuntu/Kali/Windows only if time remains	Multi-user authentication, RBAC, remote deployment

5. Priorities: what must ship

Priority	Workstream	Must produce	If cut
P0	Protection hygiene	Release build: no debug/symbols, no paths, per-build key, encoded policy table	Cannot claim protection evidence
P0	Report safety	Sanitized JSON export; no `/home`, username, key, email or token	Cannot share evidence safely
P0	One-click UI	Level 5 demo: analyze → protect → compare → export	Demo is slow and error-prone
P0	Packaging	Fedora tarball + `install.sh` + `run.sh` + desktop entry	Reviewer cannot run it
P1	Dashboard	Content metrics: text bytes, per-function sizes, XOR, key refs, CFG, program strings	Comparison looks empty
P1	Binary hygiene	Any-ELF metadata profile, UI label, tests; no execution	Binary-only story is missing
P1	SAST/DAST	Semgrep/Bandit/gitleaks + ZAP baseline + API tests	Security story is only theoretical
P1	Jobs/storage	Size column, total, delete, export menu	Disk usage remains opaque
P2	Podman/AppImage	Container or portable image	Nice-to-have
P2	Anti-tamper/anti-debug	Integrity check or debugger detection	Future work; not required for demo
P2	Dynamic binary analysis	Sandboxed trace of samples	Explicitly out of scope; design only

6. Three-day schedule

The schedule assumes **8 focused hours per day**. If only 5–6 hours are available, keep P0 and drop the P2 column; do not compress the Day 1 checkpoint. Times are suggestions, not deadlines.

Day 1 — Baseline freeze, protection hygiene, report safety, UI skeleton

Time	ID	Task	Output and acceptance
08:30–09:00	D1-01	Freeze baseline: backup `runs/` index, run all tests, record versions	`docs/baseline.txt`; 54 tests complete (1 skipped); project ZIP

Time	ID	Task	Output and acceptance
09:00–11:00	D1-02	Release artifact hygiene: `-g0`, `strip --strip-unneeded`, remove `.comment`/`.annobin.notes`, prefix map	`level-5-protected-release.elf`; `readelf` shows no `.debug_*`/`.symtab`; `strings` has no `/home`
11:00–13:00	D1-03	Protection v2: per-build key, split key shares, encoded policy table for 1/2/3/99; update tests	Two builds have different keys; no plaintext policy values; sample still compiles
13:00–13:30	D1-03b	Binary hygiene verification: run on any ELF copy; check no debug/symtab/paths; UI label	54 tests pass; artifact 0600; original unchanged
13:30–15:00	D1-04	Report sanitizer: recursive redaction, CLI, tests	`sanitize_report.py`; 0 `/home`, 0 key, 0 email/token in fixture; hashes preserved
15:00–17:00	D1-05	One-click UI skeleton: default Level 5, all tools, progress overlay, auto-compare	One click runs analyze + protect + compare; cancel works
17:00–18:00	D1-06	Integration checkpoint: tests, screenshot, commit/backup	All tests pass; demo path recorded; no regressions

Day 2 — Usable UI, security analysis, packaging foundation

Time	ID	Task	Output and acceptance
09:00–11:00	D2-01	Comparison dashboard + content metrics endpoint	Cards/table show `.text`, per-function size, XOR, key refs, CFG, program strings; no score
11:00–12:30	D2-02	Live progress and Monitor fixes: per-tool status, filters, cancel, row spacing	No overlapping labels; errors pinned; cancel stops tools
12:30–14:00	D2-03	Jobs/storage/export: size column, total, delete, sanitized export, artifact download	UI size matches `du`; delete asks for confirmation; sanitized file opens
14:00–16:00	D2-04	SAST: Semgrep, Bandit, detect-secrets/gitleaks; triage and fix high/medium	`docs/security/*.json`; no unaddressed high/medium; low findings explained
16:00–18:00	D2-05	DAST: ZAP baseline + security headers + API negative tests	`zap-baseline.html/json`; headers present; 403/413/415 tests pass
18:00–19:00	D2-06	Packaging skeleton: `install.sh`, `run.sh`, `.desktop`, `VERSION`, `CHECKSUMS.txt`	Run from project folder; browser opens; no root

Day 3 — Release, evidence, rehearsal, submission

Time	ID	Task	Output and acceptance
09:00–11:00	D3-01	Clean Fedora test: extract tarball, install, run, smoke test, uninstall	No root; no system install; browser opens; `runs/` writable; tests pass
11:00–12:30	D3-02	End-to-end demo and evidence capture: Level 5 + optional EasyEffects	Screenshots, sanitized report, artifact hashes, timing
12:30–14:00	D3-03	Finalize Word documents and PDFs: metrics, screenshots, security results, no placeholders	PDF page check: cover, dense tables, final page; links resolve

Time	ID	Task	Output and acceptance
14:00–15:00	D3-04	Demo rehearsal: 7-minute script, fallback path, Q&A notes	Under 8 minutes; no dead ends; fallback ready
15:00–16:00	D3-05	Buffer and submission: checksums, ZIP, upload, final checklist	`CHECKSUMS.txt` verifies; submission link/file confirmed

7. P0 playbooks

7.1 Release artifact hygiene (D1-02)

The current protected build is compiled with `-g` and is not stripped. That is the opposite of artifact hygiene. Keep a debug build only for internal analysis; ship a release build that carries no source map.

```
# Build the transformed sample, then create a release artifact.
gcc -O1 -std=c11 -fno-asynchronous-unwind-tables \
   -fdebug-prefix-map="$PWD"=. \
   -I protected/transformed-src \
   protected/transformed-src/main.c protected/transformed-src/rules.c \
   -o protected/level-5-protected.elf

strip --strip-unneeded -o protected/level-5-protected-release.elf \
   protected/level-5-protected.elf

objcopy --remove-section=.comment \
      --remove-section=.annobin.notes \
      --remove-section=.gnu.build.attributes \
      protected/level-5-protected-release.elf

readelf -S protected/level-5-protected-release.elf | grep -E 'debug|symtab' || echo "no debug/symtab"
nm protected/level-5-protected-release.elf 2>&1 | head
strings -a protected/level-5-protected-release.elf | grep -E '/home|transformed-src' || echo "no local paths"
```

> **Acceptance**
>
> `readelf` shows no `.debug_*` and no `.symtab`; `nm` reports no symbols; `strings` has no `/home` or `transformed-src`; the file still has a valid ELF header and the same entry point. Do not execute it.

7.2 Protection v2: per-build key and encoded policy table (D1-03)

The current transformer blinds only literals written with a `u` suffix and skips lines containing `enum`. That is why `1`, `2`, `3` and `99` survive. The v2 design must encode the **policy table**, not just two numbers.

```
/* One key per build, split into two shares. The symbol is hidden and stripped. */
/* rs_key.c */
#include <stdint.h>
volatile uint32_t rs_k1 = 0x51A7C3E9u;   /* generated per build */
volatile uint32_t rs_k2 = 0x8E2B4D10u;   /* generated per build */
uint32_t rs_key(void) { return rs_k1 ^ rs_k2; }

/* Policy table encoded at build time; decoded only at runtime. */
/* K(x) = x ^ key, generated by the Python transformer. */
static const uint32_t rs_min_role[4] = { K(1), K(2), K(3), K(99) };

uint32_t min = rs_min_role[resource & 3u] ^ rs_key();
```

The table values are generated by Python using the same per-build key. The C source never contains the plaintext policy values; the compiled .rodata contains only encoded words. This is still reversible once the key is recovered, but it removes the trivial cmp $0x2 / cmp $0x3 / mov $0x63 map.

> **Honesty note**
> A determined analyst can still recover the key from .data. The claim is **not** 'unbreakable'; it is 'the obvious constants are no longer the first thing an analyst sees, and we measure the difference'. Do not add a security score.

7.3 Report sanitizer (D1-04)

The current reports contain absolute paths, commands, Ghidra logs, the demo key, and CA paths. A submission copy must be safe to share while preserving hashes and tool versions for reproducibility.

Data class	Examples in current reports	Sanitized output
Local identity	`/home/fol3ds`, username, project root	`<HOME>`, `<USER>`, `<PROJECT>`
Tool commands	Absolute input/output paths in `command`	Replace only path arguments; keep tool and flags
Demo secret	`"key": "0x9E3779B9"`	`<DEMO_KEY>` or omit; keep method and strength
Personal data	Email, IPv4/IPv6 (except 127.0.0.1), names	`<EMAIL>`, `<IP>`
Secrets	Token-like strings, PEM blocks, cloud keys	`<REDACTED_SECRET>`
Evidence	SHA-256, sizes, counts, versions, status	Unchanged

Implement as a recursive walk over the report object. Add a CLI (python3 sanitize_report.py in.json out.json) and a UI action. Tests must assert: no /home/, no username, no 0x9E3779B9, no email, no ghp_/sk-/AKIA pattern; hash fields unchanged; output is valid JSON; running the sanitizer twice is idempotent.

7.4 One-click demo flow (D1-05 / D2-01)

1. Open the workbench and land on **Demo** (or a prominent **Run Level 5 demo** button in Operate).
2. Defaults: bundled **Level 5 · Policy Engine**, all available tools, profile **Demo constant blinding**, authorization checked.
3. Click **Run demo**: the UI creates the selection, starts protect_compare, and shows a 4-phase progress bar.
4. On completion, open **Compare** automatically: cards, content table, warnings, and download buttons.
5. Click **Export sanitized report**: download JSON + a one-page HTML/Markdown summary; show the path and size.
6. Keep **Cancel** available at every phase; a cancelled job must leave the original snapshot and baseline report intact.

> **Acceptance**
> A first-time reviewer can complete the flow without reading the README, in under 3 minutes, and can explain what changed using the dashboard. No target is executed. The raw report remains available behind a warning.

7.5 Binary metadata hygiene (implemented)

This profile works on a copy of any selected ELF without source. It is not a rewriter and not an obfuscator; it removes metadata that makes navigation cheap. The implementation is in binary_protection.py and is exposed as the binary_hygiene profile.

```
strip --strip-all binary-hygiene.elf
```

```
objcopy --remove-section=.comment binary-hygiene.elf
objcopy --remove-section=.annobin.notes binary-hygiene.elf
objcopy --remove-section=.gnu.build.attributes binary-hygiene.elf
objcopy --remove-section=.note.gnu.build-id binary-hygiene.elf
```

Check	Expected
Input	Any regular ELF ≤ 64 MiB; original unchanged
Output	`binary-hygiene/binary-hygiene.elf`, mode 0600, not executable
Metadata	No `.symtab`, `.debug_*`, `.comment`, `.annobin.notes`, `.gnu.build.attributes`, `.note.gnu.build-id`
Paths	No `/home` or `transformed-src` in `strings`
Report	`binary_metadata_hygiene_applied`, `metadata_only`, `behavior_verified: false`
Boundary	No execution; no code rewriting; no packing

Do not overclaim

Binary hygiene reduces annotations. It does not stop a decompiler. Use it as a binary-only baseline and label it metadata_only; keep instruction rewriting and runtime packing in Future Work.

8. UI upgrade design

8.1 Current pain points

Screen	Observation	Fix
Operate	Strong visual language, but the path from source to protection is long	Add a demo stepper and safe defaults
Compare	Empty until a job is selected; hint only	Add 'Open latest comparison' and 'Run demo' actions
Monitor	Job metadata runs together (`Engineprotect_compare · completed…`)	Use a grid: label, operation, status, ID, time; wrap cleanly
Storage	No size or delete UI	Add size column, total, refresh, delete with confirmation
Export	Only full JSON; contains paths and key	Sanitized export default; raw behind warning

8.2 Target flow



Figure 1. Target demo flow: select → snapshot → analyze → transform sample copy → reanalyze → compare → export.

8.3 Comparison dashboard

Replace the single 'Basic metadata & differences' view with two layers: a **summary** of security-relevant content and **details** for each artifact class. Do not present a single 'protection score'.

Card / row	What it shows	Why it matters
Artifact	Size, `.text` bytes, sections, SHA-256	Shows whether the build changed at all

Card / row	What it shows	Why it matters
Symbol surface	ELF `.symtab` entries, `nm` output, Ghidra functions	Separates real symbols from Ghidra labels
Policy content	Plaintext 1/2/3/99 present? encoded table present?	Directly answers 'did the policy table disappear?'
Key exposure	`rs_key` refs, key bytes in `.data`, key uniqueness	Makes the weakness visible, not hidden
Control flow	CFG nodes/edges, per-function sizes, XOR/guard branches	Shows structural cost
Program strings	Six policy messages, source paths, debug paths	Separates program text from metadata noise
Ghidra	Time, functions, pseudocode bodies, recovered logic	Shows analyst cost, not just file delta

8.4 Live progress and job/storage management

- Per-tool status list: queued, running, ok, tool_error, timeout, cancelled; elapsed time; output bytes.
- Progress is phase-based (snapshot → analyze → transform → reanalyze → compare), not a fake percentage.
- Pinned error panel; filter by tool/phase; auto-scroll toggle; copy log button.
- Jobs table: label, operation, status, started, duration, size on disk, actions (Open, Export, Delete).
- Storage header: total runs/ size, count, last refresh; delete requires typing the job ID or a confirmation dialog.
- Export menu: sanitized JSON (default), full JSON (warning), HTML summary, protected/stripped artifact.

8.5 Usability and accessibility acceptance

Check	Pass condition
First-run	A new user reaches a completed comparison in under 3 minutes
Keyboard	All controls reachable by Tab; visible focus; Escape closes dialogs
Labels	Every input has a label; error text is specific and next to the field
Responsive	At 1280 px and 1440 px, tables scroll; no horizontal page scroll
State	Disabled buttons explain why; loading and empty states are explicit
Truthfulness	Every screen says static-only and 'no security score' where relevant

9. Packaging: download and run on Fedora

The reviewer should not need a Python tutorial, a Ghidra install, or root. The recommended P0 is a Fedora tarball with a small installer and a launcher. A container is a P1/P2 alternative; an AppImage is P2.

Option	Effort	Size / dependency	Recommendation
Fedora tarball + `install.sh` + `run.sh`	Low–medium	Project + optional `tools/` (1.2 GB offline); Python 3.11+	**P0 — ship this**
Lite tarball, fetch tools on first run	Low	~2 MB; network + SHA-256 verification	P0 alternative for small upload
Podman/Docker image	Medium	Image ~1.2 GB; Podman rootless	P1 if time; host networking for loopback
AppImage	High	Python + app; Ghidra external; 50–100 MB	P2; not on the critical path

Option	Effort	Size / dependency	Recommendation
Windows `.exe`	High	Python + tools; cross-platform paths	Out of scope for this window

9.1 Recommended bundle layout

```
ReverseShield-1.0-fedora/
 ├── app.py, analysis.py, workflow.py, engine.py, ...
 ├── static/              # UI
 ├── samples/             # sources + built ELF samples
 ├── tools/               # optional offline JDK 25 + Ghidra 12.2
 ├── docs/                # this plan, report, screenshots
 ├── install.sh           # checks, venv, desktop entry, smoke test
 ├── run.sh               # start server, open browser, trap Ctrl+C
 ├── uninstall.sh         # remove desktop entry/state only
 ├── reverseshield.desktop   # launcher
 ├── VERSION
 ├── CHECKSUMS.txt
 └── README-FIRST.txt       # 5-step quick start
```

9.2 Installer behaviour

1. Verify Fedora 43 (warn on other distributions), Python 3.11+, `python3 -m venv`, `xdg-open`, and `sha256sum`.
2. Create a project-local `.venv` (the app uses the standard library; no pip install is required for the core).
3. If `tools/jdk25/bin/java` or Ghidra is missing, either fail with a clear message (offline bundle) or fetch and verify the pinned archives (lite bundle).
4. Build sample ELFs only if missing (`python3 build_samples.py`); never execute them.
5. Create `~/.local/share/applications/reverseshield.desktop` and an icon; do not write to `/usr` or require sudo.
6. Run `python3 -m unittest discover -s tests` and print a short result; stop on failure.

9.3 Run and uninstall

```
# Install (no root)
tar -xzf ReverseShield-1.0-fedora.tar.gz
cd ReverseShield-1.0-fedora
./install.sh

# Run
./run.sh
# -> starts 127.0.0.1:8765, opens the browser, logs to ~/.local/state/reverseshield/

# Uninstall (keeps the project and runs/)
./uninstall.sh
```

9.4 Clean-machine test

Test	Command / check	Pass condition
Fresh extract	`tar -xzf` into an empty directory	No missing files; checksums verify
No root	Run as normal user; `sudo` not used	Install completes; no writes to `/usr`
Offline	Disconnect network; run demo	Level 5 demo completes with bundled tools

Test	Command / check	Pass condition
Loopback only	`ss -ltnp | grep 8765`	Listens on `127.0.0.1`, not `0.0.0.0`
Smoke	`./run.sh`, open browser, run Level 5 demo	Comparison visible; export downloads
Uninstall	`./uninstall.sh`	Desktop entry removed; project intact

10. SAST, DAST, and data-leak prevention

10.1 SAST baseline (preliminary, run before the plan)

A preliminary scan was run on the project Python and JavaScript files (excluding tools/, runs/, backups/). It is a baseline, not the full Day 2 review. Results: **Bandit 1.9.4 — 18 low, 0 medium, 0 high**; **Semgrep 1.184.0 — 0 findings** (security-audit, secrets, Python, JavaScript packs); **detect-secrets 1.5.0 — 0 findings**. The Bandit findings are B404 (subprocess import) and B603 (subprocess call) — expected for a tool orchestrator; all calls use argv lists, shell=False, validated targets, and bounded timeouts.

Tool	Scope	Result	Action
Bandit 1.9.4	11 Python modules, 1,965 LOC	18 low (B404/B603); 0 medium/high	Document; add `# nosec` only with justification; keep argv lists
Semgrep 1.184.0	Python + JS	0 findings	Re-run with project-specific rules; add C sample rules
detect-secrets 1.5.0	Project files, excluding tools/runs	0 findings	Add gitleaks to CI; scan before export
Ghidra/binutils	Tool versions	Ghidra 12.2; binutils 2.45.1	Record versions in reports

10.2 SAST commands for Day 2

```
# Bandit (project-local venv)
python3 -m venv .venv-security
.venv-security/bin/pip install bandit semgrep detect-secrets
.venv-security/bin/bandit -r app.py analysis.py workflow.py engine.py \
   discovery.py rpm_ingest.py demo_protection.py http_upgrade.py \
   -f json -o docs/security/bandit.json

# Semgrep (specific rule packs; --config auto is not used with metrics off)
.venv-security/bin/semgrep scan --config p/security-audit --config p/secrets \
   --config p/python --config p/javascript --metrics=off \
   --json --output docs/security/semgrep.json \
   app.py analysis.py workflow.py engine.py discovery.py rpm_ingest.py \
   demo_protection.py http_upgrade.py static/workbench.js

# Secrets (project-local; exclude tools/, runs/, backups/)
.venv-security/bin/detect-secrets scan --all-files \
   --exclude-files 'tools/.*|runs/.*|backups/.*' . \
   > docs/security/detect-secrets.json
```

10.3 DAST: multiple methods for the local workbench

DAST here means testing **your own local workbench** over HTTP, not scanning a third-party application. The server is loopback-only; use --network=host so a container can reach 127.0.0.1. ZAP baseline is the P0 method; the other rows are alternatives or stretch methods to document in the report.

Method	Command (example)	Coverage	Limits
ZAP baseline (passive)	`docker run --rm --network=host -v "$PWD/docs/security:/zap/wrk:Z" ghcr.io/zaproxy/zaproxy:stable zap-baseline.py -t http://127.0.0.1:8765 -r zap-baseline.html -J zap-baseline.json`	Headers, cookies, passive rules, spider	No authenticated state-changing requests; baseline is not a full pentest
ZAP API scan	`zap-api-scan.py -t openapi.json -f openapi`	Endpoint coverage, request/response checks	Needs an OpenAPI spec; write a minimal spec as P2
Nuclei	`nuclei -u http://127.0.0.1:8765 -severity low,medium,high,critical -o nuclei.txt`	Known templates, misconfigurations	Template coverage is generic; not app-specific
Nikto	`docker run --rm --network=host sullo/nikto -h http://127.0.0.1:8765`	Server misconfigurations, files	Noisy; limited SPA coverage
Manual proxy (ZAP/Burp)	Configure browser proxy + CA; explore Operate/Compare/Monitor	Auth token, origin checks, upload flow	Manual; needs a checklist
Custom API tests	`curl` matrix: no token, foreign Origin, oversized upload, non-ELF, path traversal	Business logic and input validation	Only the cases you write

ZAP scope and safety

Run only against http://127.0.0.1:8765 while you own the process. Do not use active attack mode against third-party sites. Stop the server after the scan. Keep the raw ZAP report internal; export a sanitized summary with the submission.

10.4 API negative tests

Test	Request	Expected
Missing token	`POST /api/jobs` without `X-ReverseShield-Token`	403
Foreign origin	`Origin: https://evil.example` + valid token	403
Oversized upload	`POST /api/ingest` > 64 MiB	413
Wrong magic	Upload `PK\x03\x04`	415
Chunked body	`Transfer-Encoding: chunked`	400
Traversal filename	`filename=../../etc/passwd`	Stored basename only; no path escape
Unknown job	`GET /api/jobs/deadbeef`	404
Report before finish	`GET /api/jobs/<id>/report`	409

10.5 Data-leak prevention

- Classify report fields: public (counts, versions), internal (paths, command lines), sensitive (key, tokens, PII).
- Default export = sanitized JSON + HTML summary; full JSON requires an explicit warning and is never the demo download.
- Redact home directories, usernames, project roots, emails, non-loopback IPs, tokens, PEM blocks, and the demo key.
- Keep SHA-256, sizes, tool versions, counts, status and timing — they are the reproducibility evidence.
- Add a UI badge: 'This report contains local paths' with a **Sanitize** action.
- Scan the repo and docs/ with gitleaks/detect-secrets before submission; add a pre-commit hook if time.
- Never store uploaded file bytes in the report; store hashes, sizes, and parsed metadata only.
- Keep runs/ at 0700 and files at 0600; document that runs/ is local evidence, not a public share.

11. Evidence pack and demo script

11.1 What to capture

Evidence	File / format	Why
Baseline test run	`docs/evidence/tests.txt`	Shows the suite is green before and after
Level 5 comparison	Screenshot + `report.sanitized.json`	Main demo evidence
Release artifact checks	`readelf`, `nm`, `strings` output	Proves artifact hygiene
Key uniqueness	Two builds + SHA-256 + key values	Proves per-build diversity
SAST	Semgrep/Bandit/detect-secrets JSON	Shows static review
DAST	ZAP HTML/JSON + API negative-test log	Shows HTTP testing
Packaging	Install transcript + desktop entry + `ss` output	Proves download-and-run
Limitations	One page in both Word docs	Prevents overclaiming

11.2 Seven-minute demo script

1. **0:00–0:40 — Frame the problem.** Symbols, debug paths, and policy constants make reverse engineering cheap; the project measures what a transformation actually changes.
2. **0:40–1:30 — Show the app.** Open the installed desktop entry; point to 'static only' and 'no target execution'.
3. **1:30–3:00 — One-click demo.** Run Level 5; show live phases and tool status; note the snapshot path.
4. **3:00–4:30 — Compare.** Show .text, per-function sizes, XOR/key refs, policy values, CFG, program strings; explicitly say 'not a score'.
5. **4:30–5:30 — Sanitized export.** Download; show no /home, no key; mention the raw report is internal.
6. **5:30–6:30 — Security evidence.** Show Bandit/Semgrep/ZAP results and the one or two low findings with disposition.
7. **6:30–7:00 — Limits and next step.** Demo-grade protection, no execution, future dynamic sandbox and anti-tamper.

12. Submission checklist

Area	Check	Done
Code	All tests pass; `py_compile`; `node --check`; no debug prints	☐
Protection	Release build has no debug/symbols/paths; key differs per build; policy table encoded	☐
Reporting	Sanitized export has no `/home`, username, key, email, token; hashes preserved	☐
UI	One-click demo completes; Compare populated; Monitor readable; storage/delete works	☐
Packaging	Fresh Fedora extract → install → run → demo → uninstall; no root	☐
SAST	Bandit/Semgrep/detect-secrets reports; no unaddressed medium/high	☐
DAST	ZAP baseline + API negative tests; scope and limits documented	☐

Area	Check	Done
Documents	Two DOCX + PDF; cover, TOC updated, page numbers, no placeholders	☐
Evidence	Screenshots, hashes, checksums, demo recording (optional)	☐
Submission	ZIP/tarball + checksums + link/file confirmed	☐

13. References

[1] OWASP Foundation. OWASP Web Security Testing Guide. https://owasp.org/projects/web-security-testing-guide

[2] ZAP. Baseline Scan. https://www.zaproxy.org/docs/docker/baseline-scan/

[3] ZAP. API Scan. https://www.zaproxy.org/docs/docker/api-scan/

[4] Semgrep. Supported languages. https://docs.semgrep.dev/supported-languages

[5] Bandit. Getting Started. https://bandit.readthedocs.io/en/latest/start.html

[6] Gitleaks. Secret scanning. https://github.com/gitleaks/gitleaks

[7] TruffleHog. Documentation. https://github.com/trufflesecurity/trufflehog

[8] ProjectDiscovery. Running Nuclei. https://docs.projectdiscovery.io/tools/nuclei/running

[9] GNU Binutils. strip. https://sourceware.org/binutils/docs/binutils/strip.html

[10] Ghidra. Software reverse engineering suite. https://ghidra-sre.org/

[11] OWASP. Top 10 (2021). https://owasp.org/Top10/

[12] MITRE. CWE-200: Exposure of Sensitive Information. https://cwe.mitre.org/data/definitions/200.html

[13] MITRE. CWE-798: Use of Hard-coded Credentials. https://cwe.mitre.org/data/definitions/798.html

[14] Podman. Documentation. https://podman.io/docs

[15] AppImage. Documentation. https://docs.appimage.org/

Appendix A — Commands and file map

A.1 Baseline and verification

```
cd /home/fol3ds/fol3ds/personal/reverseshield-local-demo
python3 -m unittest discover -s tests
python3 -m py_compile app.py analysis.py workflow.py engine.py \
  discovery.py rpm_ingest.py demo_protection.py http_upgrade.py
node --check static/workbench.js
RUN_GHIDRA_TESTS=1 python3 -m unittest discover -s tests -p 'test_engine.py' -v
```

A.2 Storage map

```
runs/<job_id>/
  ├── original-0.elf           # snapshot of the selected target (0600)
  ├── original-N.elf           # more targets, if selected
  ├── events.jsonl             # live event stream (bounded)
  ├── report.json              # final report (full)
  ├── analysis-<hex>/          # analyze jobs; Ghidra export/logs
  ├── before-<hex>/            # compare jobs: baseline analysis
  ├── after-<hex>/             # compare jobs: reanalysis
  ├── symbols-removed.elf      # strip_compare artifact
  └── protected/
      ├── *-protected.elf      # demo protection artifact
      └── transformed-src/     # transformed C sources + headers
```

A.3 API routes used by the UI

Method	Route	Purpose
GET	/api/status	Version, tools, mode, limits, token
GET	/api/jobs	Session job list
GET	/api/jobs/<id>?after=N	Events after sequence N
GET	/api/jobs/<id>/report	Download full report
GET	/api/jobs/<id>/artifact	Download stripped/protected artifact
POST	/api/discover	Sample, Flatpak, or local path discovery
POST	/api/ingest	Upload ELF/RPM bytes
POST	/api/jobs	Start analyze / strip_compare / protect_compare
POST	/api/jobs/<id>/cancel	Request cancellation